

DOCUMENT D'ANALYSE ET **DE CONCEPTION**

Projet : implémentation de la commande
myls

(Commande originale reproduite : *ls -
list directory contents*)

Membres de l'équipe 6 :

- ☐ BITOUKOA Ousmane Lamé
- ☐ AROUNG Ralph Stéphane Darel
- ☐ MUKETE Mildred Esimo
- ☐ NOUMBO DONANG Michel-Ange
- ☐ JEAN-JOREST NDE MU FODJO

Sous la supervision de : Emmanuel NGUIMBUS

1. Introduction et contexte

1.1. Présentation générale du projet

Dans le cadre du cours de Théorie des Systèmes d'Exploitation dispensé en troisième année de la filière SRT (Systèmes, Réseaux et Télécommunications) à Saint Jean Ingénieur, les étudiants ont pour but de reproduire, en langage C des commandes fondamentales du système d'exploitation Linux. Ce projet vise à consolider les acquis théoriques en nous confrontant à des problèmes concrets de programmation système. À cet effet, notre équipe a été chargée d'implémenter la commande `mys`, une reproduction fidèle de la commande `ls` native de Linux. La commande `ls` (list) est l'une des commandes les plus utilisées dans tout environnement Unix/Linux : elle permet de lister le contenu d'un ou de plusieurs répertoires, avec de nombreuses options d'affichage et de filtrage

1.2. Objectifs pédagogiques

Ce projet poursuit les objectifs pédagogiques suivants :

- ❖ Mettre en oeuvre les concepts fondamentaux des systèmes d'exploitation
- ❖ Manipuler les appels systèmes POSIX liés aux fichiers et répertoires : `opendir`, `readdir`, `closedir`, `stat`, `lstat`, `getpwnuid`, `getgrgid`, et les structures associées
- ❖ Développer une application C structurée et modulaire, en appliquant les bonnes pratiques de développement logiciel
- ❖ Travailler en équipe sur un projet de développement complet, de l'analyse fonctionnelle jusqu'aux tests de validation, en passant par la conception et l'implémentation
- ❖ Utiliser des outils professionnels de gestion de version (Git/GitHub) pour collaborer de manière coordonnée

2. Analyse fonctionnelle

2.1 Description de la commande originale ls

La commande `ls` (abréviation de `list`) est l'une des commandes fondamentales des systèmes UNIX/Linux. Son rôle principal est de lister le contenu d'un ou plusieurs répertoires du système de fichiers, en affichant les noms des fichiers et, selon les options activées, leurs métadonnées associées.

Concrètement, `ls` interagit avec le noyau du système d'exploitation via des appels système (syscalls) pour :

- Ouvrir un répertoire grâce à `opendir()` / `readdir()` / `closedir()`
- Lire les entrées du répertoire (fichiers, sous-répertoires, liens symboliques...)
- Récupérer les métadonnées de chaque entrée via l'appel `stat()` ou `lstat()`
- Formater et afficher les résultats selon les options choisies par l'utilisateur

Sans aucune option, `ls` se comporte de la manière suivante :

- Il liste le contenu du répertoire courant (.) si aucun chemin n'est précisé
- Les fichiers et dossiers sont affichés en colonnes, triés par ordre alphabétique
- Les fichiers cachés (dont le nom commence par un point .) sont exclus de l'affichage
- Seuls les noms sont affichés, sans aucune métadonnée supplémentaire

La commande `ls` exploite les informations retournées par `stat()` pour chaque entrée. Ces métadonnées incluent :

Métadonnée	Description
Types et permissions	Type de fichier (ordinaire, répertoire, lien...) et droits d'accès (rwxrwxrwx)
Nombre de liens durs	Nombre de références physiques vers l'inode
Propriétaire / Groupe	Nom d'utilisateur et groupe associés au fichier
Taille	Taille du fichier en octets
Date de modification	Horodatage de la dernière modification du contenu
Nom du fichier	Nom de l'entrée dans le répertoire

L'implémentation de `myls` repose principalement sur les appels système POSIX suivants :

- `opendir()` : Ouvre un flux de lecture vers le répertoire cible
- `readdir()` : Lit séquentiellement chaque entrée du répertoire (`struct dirent`)
- `stat()` / `lstat()` : Récupère les métadonnées d'un fichier. `lstat()` ne suit pas les liens symboliques
- `closedir()` : Ferme le flux ouvert par `opendir()` et libère les ressources

2.2 Inventaires des options à implémenter

Le tableau ci-dessous présente l'ensemble des options de la commande *ls* à implémenter dans *myls*, conformément aux exigences du sujet. Chaque option modifie le comportement par défaut de l'affichage.

Option	Nom / Effet	Comportement dans <i>myls</i>
-a	All – Afficher tous les fichiers	Inclure les entrées commençant par un point (.) dans l'affichage, notamment . et ..
-A	Almost-all	Afficher les fichiers cachés sauf . et ..
-l	Long format – Affichage détaillé	Afficher permissions, liens, propriétaire, groupe, taille, date et nom
-R	Recursive – Récursif	Lister récursivement le contenu de tous les sous-répertoires
-t	Time – Tri par date	Trier les fichiers par date de dernière modification (plus récent en premier)
-r	Reverse – Ordre inverse	Inverser l'ordre de tri (alphabétique ou par date selon options)
-S	Size – Tri par taille	Trier les fichiers par taille décroissante
-h	Human-readable	Avec -l, afficher les tailles dans un format lisible (K, M, G)
-i	Inode	Afficher le numéro d'inode de chaque entrée
-n	Numeric UID/GID	Avec -l, afficher les UID et GID numériques au lieu des noms

-d	Directory	Afficher les répertoires eux-mêmes (et non leur contenu)
-F	Classify	Ajouter un indicateur après chaque nom : / pour répertoire, * pour exécutable, @ pour lien symbolique
-p	Slash pour répertoires	Ajouter / après les noms de répertoires
-l	One per line	Afficher un fichier par ligne
-C	Colonnes (défaut)	Forcer l'affichage en colonnes (par défaut sur terminal)
-m	Comma-separated	Afficher les entrées séparées par des virgules
-s	Size en blocs	Afficher la taille allouée en blocs pour chaque fichier
-u	Access time	Utiliser la date de dernier accès au lieu de la date de modification
-c	Change time	Utiliser la date de dernier changement de statut (ctime)
-g	Long sans propriétaire	Comme -l mais sans afficher le propriétaire
-o	Long sans groupe	Comme -l mais sans afficher le groupe
-G	No-group	Ne pas afficher le groupe dans l'affichage long
-L	Dereference liens	Suivre les liens symboliques (afficher les infos de la cible)
-H	Dereference args	Suivre les liens symboliques uniquement pour les arguments de la ligne de commande

<code>--color</code>	Colorisation	Activer la colorisation de la sortie (type de fichier)
----------------------	--------------	--

Note : les options peuvent être combinées (ex. : `-la`, `-ltr`, `-lRa`). L'ordre des options n'a aucune incidence sur le résultat final.

2.3 Syntaxe d'exécution

La commande `mys` respecte la syntaxe UNIX standard suivante :

```
mys [options] [chemin...]
```

Les éléments entre crochets sont facultatifs. Sans argument, `mys` liste le répertoire courant sans aucune option.

Composante	Description
<code>mys</code>	Nom de l'exécutable produit par la compilation du programme
<code>[options]</code>	Une ou plusieurs options préfixées d'un tiret (-). Peuvent être regroupées (ex. : <code>-la</code> équivaut à <code>-l -a</code>)
<code>[chemin...]</code>	Un ou plusieurs chemins (absolus ou relatifs) vers des fichiers ou répertoires à lister. Si omis, le répertoire courant <code>.'</code> est utilisé par défaut

Exemples de commandes valides :

`mys` : liste le repertoire courant, sans option

`mys -l /home/user` : affiche le contenu de `/home/user` en format long

`mys -la` : affiche tous les fichiers (y compris les cachés) du répertoire courant en format long

`myls -ltr /var/log` : affiche le contenu de `/var/log` en format long, trié par date de modification (ordre inverse, le plus ancien en premier)

`myls -R ./projet` : liste récursivement le répertoire `./projet` et tous ses sous-répertoires

`myls -lh /usr/share` : format long avec tailles lisibles (Ko, Mo...) pour `/usr/share`

Concernant la gestion des erreurs de syntaxe, le programme doit gérer proprement les cas d'utilisation incorrecte :

- Option inconnue : afficher un message d'erreur du type "option invalide : -x" et retourner un code d'erreur non nul
- Chemin inexistant : afficher "myls: /chemin: No such file or directory" et continuer avec les autres arguments
- Permission refusée : afficher "myls: /chemin: Permission denied" et passer à l'entrée suivante

3. Analyse technique

3.1 Environnement de développement

L'environnement de développement :

- Le projet myls est développé sous Linux, la commande ls originale étant un utilitaire Unix.
- Cela garantit la compatibilité avec les opérations sur les fichiers au niveau système.
- L'implémentation est réalisée en langage C, permettant une interaction directe avec les appels système et une gestion efficace de la mémoire.
- La compilation du code source est effectuée avec GCC (GNU Compiler Collection).
- Le développement s'effectue à l'aide d'un éditeur de texte ou d'un environnement de développement intégré (IDE).
- Le contrôle de version est assuré par Git, et le projet est hébergé sur GitHub afin de faciliter la collaboration entre les membres de l'équipe.

3.2 Librairies et appels système

La mise en œuvre de myls repose sur plusieurs bibliothèques C standard et appels système :

- *<dirent.h>* : Cette bibliothèque gère les opérations sur les répertoires. Des fonctions telles que `opendir()` et `readdir()` permettent d'ouvrir un répertoire et d'en parcourir le contenu. Elles sont utilisées lors de l'affichage des fichiers.
- *<sys/stat.h>* : Cette bibliothèque permet d'accéder aux métadonnées des fichiers via des fonctions comme `stat()` et `lstat()`. Elle est utilisée pour implémenter des options telles que `-l`, lorsque des informations détaillées sur les fichiers (taille, permissions, horodatage) sont requises.
- *<pwd.h>* et *<grp.h>* : Ces bibliothèques convertissent les identifiants utilisateur (UID) et les identifiants de groupe (GID) en noms lisibles. Elles sont nécessaires pour afficher

les informations du propriétaire et du groupe sous forme de liste détaillée.

Ces bibliothèques sont utilisées après l'analyse des arguments de la ligne de commande et lors du traitement de chaque fichier du répertoire.

3.3 Limites et contraintes

L'une des principales contraintes de ce projet est de reproduire le comportement de la commande *ls* originale aussi fidèlement que possible. Les performances sont également un facteur important, notamment lors de la manipulation de grands répertoires ou de parcours récursifs. L'utilisation efficace de la mémoire et les algorithmes de tri doivent être pris en compte. De plus, le programme est conçu pour fonctionner sous Linux, car il dépend d'appels système spécifiques à Linux.

3.4 Gestion des erreurs

Le programme doit gérer différents types d'erreurs pour garantir sa robustesse :

- Si un répertoire n'existe pas, le programme doit afficher un message d'erreur approprié.
- Si l'utilisateur n'a pas l'autorisation d'accéder à un fichier ou un répertoire, un message « Permission refusée » doit s'afficher.
- Si un fichier est inaccessible ou illisible, le programme doit l'ignorer et poursuivre son exécution.
- Une gestion correcte des erreurs garantit que le programme se comporte comme la commande *ls* d'origine et évite les plantages inattendus.

3.5 Tableau récapitulatif des appels système et des bibliothèques pour myls

Le tableau suivant récapitule les appels système et les bibliothèques utilisés dans l'implémentation de la commande *myls*, ainsi que leur objectif et leur utilisation au sein de l'algorithme.

Librairies / Appels système	Ce que cela fait	Pourquoi on l'utilise	Lorsqu'il est utilisé dans l'algorithme
<dirent.h> → opendir()	Ouvre un répertoire	Accéder au contenu du répertoire	Après l'analyse des arguments
<dirent.h> → readdir()	Lire chaque fichier dans un répertoire	Lister tous les fichiers	Pendant la boucle de parcours de répertoire
<dirent.h> → closedir()	Fermer un répertoire	Libérer les ressources	Après avoir lu tous les fichiers
<sys/stat.h> → stat()	Obtenir les informations du fichier	Récupérer la date, les permissions, la taille	Avant d'afficher les infos d'un fichier (-l)
<sys/stat.h> → lstat()	Obtenir des informations sans suivre de liens	Gérer correctement les liens symboliques	Lors de la prise en charge des fonctionnalités avancées
<unistd.h> → access()	Vérifie les permissions des fichiers	Vérifier l'accès en lecture	Avant d'accéder à un fichier
<pwd.h> → getpwuid()	Convertir UID à username	Afficher le propriétaire du fichier	Lors de la mise en forme de la sortie (-l)
<grp.h> → getgrgid()	Convertir GID à	Afficher le propriétaire du	Lors de la mise en forme de la

	groupname	groupe	sortie (-l)
<time.h> → localtime()	Convertir le format de temps	Rendre le temps lisible par l'Homme	Après l'analyse des arguments
<time.h> → strftime()	Format date/heure	Accéder au contenu d'un répertoire	Pendant la boucle de parcours du répertoire
<stdio.h> → printf()	Afficher la sortie	Lister tous les fichiers	Après avoir lu tous les fichiers
<stdio.h> → perror()	Affiche les messages d'erreur	Libérer les ressources	Avant d'afficher les informations du fichier (-l)
<stdlib.h> → malloc()	Alloue la mémoire	Récupérer la date, les permissions, la taille	Lors de la prise en charge des fonctionnalités avancées
<stdlib.h> → free()	Libérer la mémoire	Gérer correctement les liens symboliques	Avant d'accéder à un fichier
<string.h> → strcmp()	Compare les chaînes de caractère	Vérifier les accès en lecture	Lors de la mise en forme de la sortie (-l)
<string.h> → strcpy()	Copie les chaînes de caractères	Afficher le propriétaire du fichier	Lors de la mise en forme de la sortie (-l)
<dirent.h> → opendir()	Ouvre un répertoire	Afficher le propriétaire du groupe	Lors de l'affichage de la date de modification
<dirent.h> → readdir()	Lire chaque fichier dans un répertoire	Rendre le temps lisible par l'Homme	Lors de la mise en forme de la sortie

<dirent.h> → closedir()	Ferme un répertoire	Afficher la date mise en forme	Étape d'affichage final
<sys/stat.h> → stat()	Obtenir les informations du fichier	Afficher les informations/noms du fichier	Lorsqu'une erreur se produit

4. Conception détaillée

4.1 Vue globale de l'architecture

L'architecture de *mys* est organisée en modules fonctionnels indépendants, chacun responsable d'une partie bien définie du traitement. Cette répartition facilite la maintenance et la répartition des tâches au sein de l'équipe.

Module	Fichier(s)	Responsabilité
main	main.c	Point d'entrée. Analyse argc/argv, détecte options et chemins, orchestre les appels aux autres modules.
options	options.c/.h	Parse la ligne de commande. Remplit la structure t_opts. Gère les options combinées (-la = -l -a).
directory	directory.c/.h	Ouvre un répertoire, lit toutes ses entrées via opendir/readdir, construit le tableau de t_entry.
entry	entry.c/.h	Remplit les métadonnées d'une entrée (stat/lstat, getpwuid, getgrgid, résolution de lien).
sort	sort.c/.h	Contient les fonctions de comparaison et appelle qsort() selon les options actives.
display	display.c/.h	Formatage et affichage : mode simple (colonnes), mode long (-l), calcul de la largeur des colonnes.
error	error.c/.h	Affichage des messages d'erreur standardisés sur stderr avec errno.

utils	utils.c/.h	Fonctions utilitaires : formatage taille lisible, formatage permissions, concaténation de chemins.
-------	------------	---

4.2 Algorithme principal

Le déroulement général de *myls* est le suivant :

- Lire et analyser la ligne de commande (argc, argv)
- Appeler parse_options() pour remplir t_opts et obtenir la liste des chemins restants.
- Si aucun chemin n'est fourni, utiliser "." (répertoire courant) comme chemin unique.
- Pour chaque chemin fourni :
 - Appeler stat() ou lstat() pour déterminer le type (fichier ordinaire ou répertoire).
 - Si c'est un fichier ordinaire : construire directement une t_entry et l'afficher.
 - Si c'est un répertoire : appeler process_directory(path, opts).
 - Si le chemin est invalide : afficher l'erreur et passer au suivant.
- Dans process_directory(path, opts) :
 - Appeler opendir(path) – en cas d'échec, appeler handle_error() et retourner.
 - Lire toutes les entrées avec readdir() en boucle.
 - Filtrer les entrées cachées si -a n'est pas actif.
 - Pour chaque entrée valide, construire la t_entry complète (appel à fill_entry()).
 - Stocker toutes les t_entry dans un tableau dynamique.
 - Trier le tableau selon les options actives (sort_entries()).
 - Afficher le tableau selon le format choisi (display_entries()).

- Si -R est actif : pour chaque t_entry de type répertoire (sauf . et ..), appeler récursivement process_directory() avec le chemin complet.
- Libérer la mémoire allouée.
- Libérer la structure t_opts et retourner le code de retour global.

4.3 Structures de données

4.3.1 Structure t_opts – Options actives

```
typedef struct s_opts {
    int opt_a;          /* -a : afficher fichiers cachés */
    int opt_A;          /* -A : presque tous (sans . et ..) */
    int opt_l;          /* -l : format long */
    int opt_R;          /* -R : récursif */
    int opt_t;          /* -t : tri par date */
    int opt_r;          /* -r : ordre inverse */
    int opt_S;          /* -S : tri par taille */
    int opt_h;          /* -h : tailles lisibles */
    int opt_i;          /* -i : afficher inode */
    int opt_n;          /* -n : UID/GID numériques */
    int opt_d;          /* -d : répertoires comme fichiers */
    int opt_F;          /* -F : indicateur de type */
    int opt_l;          /* -l : un fichier par ligne */
    int opt_color;      /* --color : colorisation */
    /* ... autres options ... */
} t_opts;
```

Cette structure regroupe tous les drapeaux d'options sous forme d'entiers (0 ou 1). Elle est passée par pointeur à toutes les fonctions qui en ont besoin, ce qui évite les variables globales et facilite les tests unitaires.

4.3.2 Structure t_entry – Représentation d'une entrée

```
typedef struct s_entry {
    char      *name;          /* Nom de l'entrée */
    char      *path;          /* Chemin complet (pour lstat) */
    struct stat st;           /* Métadonnées (stat/lstat) */
    char      *owner_name;     /* Nom propriétaire (getpwuid) */
    char      *group_name;     /* Nom du groupe (getgrgid) */
    char      *link_target;    /* Cible du lien symbolique */
}
```



```

    char    perm_str[11];    /* Ex. : "drwxr-xr-x" */
    char    size_str[16];    /* Taille formatée (-h) ou
brute*/
    char    date_str[20];    /* Date formatée : "Mar 27
14:30"*/
} t_entry;

```

La `t_entry` centralise toutes les informations nécessaires à l'affichage d'une ligne (mode long ou simple). Précalculer les chaînes `perm_str`, `size_str` et `date_str` lors du remplissage permet de simplifier le module d'affichage et de ne pas répéter les mêmes calculs.

4.3.3 Tableau dynamique d'entrées

```

typedef struct s_dir_list {
    t_entry  *entries;    /* Tableau dynamique de t_entry*/
    int      count;        /* Nombre d'entrées actives*/
    int      capacity;     /* Capacité allouée*/
    long     total_blocks; /* Somme des blocs (pour -l)*/
} t_dir_list;

```

Le tableau est initialisé avec une capacité de 64 entrées et redimensionné par doublement (`realloc`) si nécessaire. Ce choix offre une complexité amortie $O(1)$ pour les insertions, adaptée aux répertoires de taille variable.

4.4 Algorithme de tri

Le tri des entrées est réalisé via la fonction `qsort()` de la bibliothèque standard C, ce qui garantit des performances $O(n \log n)$ dans le cas général. La fonction de comparaison est sélectionnée dynamiquement en fonction des options actives.

Options actives	Critère de tri	Fonction de comparaison
(défaut)	Nom (ordre lexicographique croissant)	<code>cmp_by_name()</code>
-t	Date de modification (plus récent en premier)	<code>cmp_by_mtime()</code>
-S	Taille (décroissante)	<code>cmp_by_size()</code>
-t -r	Date de modification (plus ancien en premier)	<code>cmp_by_mtime()</code> + inversion
-r	Nom (ordre lexicographique décroissant)	<code>cmp_by_name()</code> + inversion
-u	Date de dernier accès (<code>st_atime</code>)	<code>cmp_by_atime()</code>
-c	Date de changement de statut (<code>st_ctime</code>)	<code>cmp_by_ctime()</code>

Pseudo-code de la sélection de la fonction de comparaison :

```
int (*compare)(const void *, const void *);  
if (opts->opt_t)      compare = cmp_by_mtime;  
else if (opts->opt_S) compare = cmp_by_size;  
else if (opts->opt_u) compare = cmp_by_atime;  
else if (opts->opt_c) compare = cmp_by_ctime;  
else                 compare = cmp_by_name;
```

```
qsort(list->entries, list->count, sizeof(t_entry), compare);
```

```
if (opts->opt_r)
    reverse_entries(list->entries, list->count);
```

L'inversion est appliquée après le tri, en permutant les éléments du tableau du début à la fin. Cette approche est plus simple qu'un adaptateur de comparaison inversée et garantit le même résultat.

4.5 Gestion de la récursivité (-R)

Lorsque l'option -R est active, `process_directory()` s'appelle récursivement pour chaque sous-répertoire rencontré. Pour éviter les boucles infinies (liens symboliques circulaires), seuls les répertoires de type `DT_DIR` (ou dont `st_mode` indique `S_ISDIR()`) sont traités, et les entrées `.` et `..` sont systématiquement exclues de la récursion.

Pseudo-code de la récursion :

```
void process_directory(const char *path, t_opts *opts) {
    t_dir_list list;
    DIR *dir = opendir(path);
    if (!dir) { handle_error(path); return; }

    build_list(dir, path, opts, &list);
    closedir(dir);
    sort_entries(&list, opts);
    display_entries(&list, path, opts);

    if (opts->opt_R) {
        for (int i = 0; i < list.count; i++) {
            t_entry *e = &list.entries[i];
            if (!S_ISDIR(e->st.st_mode)) continue;
            if (strcmp(e->name, ".") == 0) continue;
            if (strcmp(e->name, "..") == 0) continue;
            char subpath[PATH_MAX];
            snprintf(subpath, sizeof(subpath),
                     "%s/%s", path, e->name);
            printf("\n%s:\n", subpath);
            process_directory(subpath, opts); /* récursion */
        }
    }
    free_list(&list);
}
```

4.6 Formatage de l'affichage long (-l)

En mode -l, chaque entrée est affichée sur une ligne selon le format suivant (identique à ls -l) :

```
-rwxr-xr-x  2  alice  staff   4096  Mar 27 14:30  myls
```

Champ	Source	Exemple
Permissions (10 car.)	mode_to_str(st.st_mode)	-rwxr-xr-x
Nombre de liens	st.st_nlink	2
Propriétaire	getpwuid(st.st_uid)->pw_name	alice
Groupe	getgrgid(st.st_gid)->gr_name	staff
Taille	st.st_size (ou formatée si -h)	4096
Date	strftime("%b %e %H:%M", localtime(&st.st_mtime))	Mar 27 14:30
Nom	entry->name (+ lien si S_ISLNK)	mysls -> /usr/bin/ls

Conversion des permissions (pseudo-code) :

```
void mode_to_str(mode_t mode, char out[11]) {
    out[0] = S_ISDIR(mode) ? 'd' : S_ISLNK(mode) ? 'l' :
             S_ISCHR(mode) ? 'c' : S_ISBLK(mode) ? 'b' :
             S_ISFIFO(mode) ? 'p' : S_ISSOCK(mode) ? 's' : '-';
    out[1] = (mode & S_IRUSR) ? 'r' : '-';
    out[2] = (mode & S_IWUSR) ? 'w' : '-';
    out[3] = (mode & S_ISUID) ? 's' :
             (mode & S_IXUSR) ? 'x' : '-';
    /* ... répéter pour groupe (S_IRGRP...) et autres (S_IROTH...)
*/
    out[10] = '\\0';
}
```

Conclusion

Ce dossier d'analyse et de conception présente l'ensemble du travail préparatoire réalisé par l'Équipe 6 pour le projet myls. L'étude approfondie de la commande ls native, la définition précise de l'architecture modulaire, la formalisation des structures de données et des algorithmes, ainsi que la planification rigoureuse des tests constituent les fondations sur lesquelles s'appuiera l'implémentation. L'objectif final est de livrer un exécutable myls robuste, fidèle au comportement de ls et compilable avec GCC sur tout système Linux POSIX. Ce projet constitue une opportunité exceptionnelle de mettre en pratique les concepts fondamentaux des systèmes d'exploitation : gestion du système de fichiers, appels système, gestion des processus et des permissions.